Chapter 7- Exercises

1)a) $A = \{s\}$ and $B = \{u, v, t\}$
$A = \{s, v\}$ and $B = \{u, t\}$
$A = \{s, v, u\}$ and $B = \{t\}$

b) It is 4, given by the cut $A = \{s, v\}$ and $B = \{t, u\}$

2)a) The value of this flow is $5+5+8 = 18$. This is not a maximum flow because there exists an augmenting s-t path. ~~(s → upper vertex, middle left vertex, middle right vertex, d, t)~~

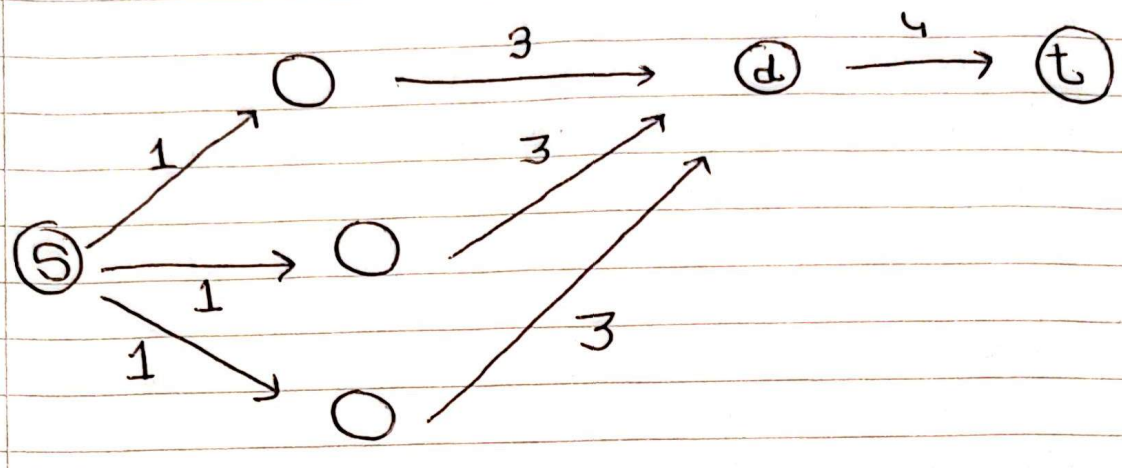
b) (s → upper → middle right → middle left → d → t).

b) ~~The~~ A minimum s-t cut is ~~$\$$~~ $A = \{s$, upper vertex$\}$ and $B = V - A$, with capacity 21.

3)a) The value of this flow is $6+3+1 = 10$. It is not a maximum flow because there exists an augmenting s-t path (s → a → c → b → d → t) with bottleneck ~~cost~~ capacity 1.

b) The minimum s-t cut is $A = \{s, a, b, c\}$ and $B = V - A$, with capacity 11.

5) The statement is false. For instance, consider this graph:-



~~Bef~~ Right now, the minimum s-t cut is $A = \{s\}$ and $B = V - A$. After incrementing each edge's capacity by 1, the minimum s-t cut becomes ~~A~~ $B = \{t\}$ and $A = V - B$.

6) We create a bipartite graph G ~~xxx~~ by letting each switch be a node in X, each fixture a node in Y, and then adding an edge from a switch to a fixture iff the line segment joining them is does not intersect with any of the walls. We also add a source s with an edge to each node in X, and a sink t with an edge from each node in Y. Every edge is given a capacity

of 1. We now use the Ford-Fulkerson algorithm to find a maximum flow in G. If the value of this flow is equal to n, we have a perfect matching in G i.e. each fixture is visible from one unique switch and so we return true. Else if the value of the flow is less than n, we return false.

Creating G takes time $O(n^2 + m)$ time, and then Ford Fulkerson takes $O(m'C)$ i.e $O(mn)$ time because $C = n$ in this case, so the overall running time is $O(n^2 m)$.
$O(|E|C)$ time, where $|E| \leq 2n + n^2$ i.e $|E| = O(n^2)$, and $C = n$ because C is the sum of capacities of edges out of s, thus the overall running time is $O(n^3 + n^2 m)$

8)a) We create a graph $G = (V, E)$ as follows. We create a set of nodes X which has a node for each blood type (so node representing bloodtype AB would be named X_{AB}) and a set of nodes Y which has a node for each bloodtype (named analogously to X's nodes). There is an

edge from a node u in X to a node v in Y if patients with v can receive u. These is edges haves capacity equal to the demand for v. We also add a source s with an edge to each node u in X and capacity equal to the supply of u, and similarly a sink t with an edge from each node v in Y, and capacity equal to the demand of v. Now we compute a maximum s-t flow using the scaling max flow algorithm. If the value of the flow is equal to the total demand of the nodes in Y, blood types represented by the nodes in Y, we return true, else we return false.

If there is a sufficient supply in which blood y gets a certain amount of blood x, then we can send a flow of that amount on the edge (x, y) while satisfying capacity conditions, and since y's demands are met, flow out of y must equals its demands and so the flow into the sink would be equal to the total demand. If there is a max flow with value equal to total demand, that means every edge into the sink is

saturated with flow, and so the flow into each ~~edge~~ node in Y is equal to ~~its~~ the demand for the blood type ~~required~~ represented by the node in Y, and so we have a sufficient supply with ~~x~~ x units of a certain blood type ~~fulfill~~ u associated with blood type v iff there ~~is~~ a edge from the node representing ~~x~~u to the node representing v ~~has~~ with flow x.

Creating the graph takes ~~O~~ O(1) time because the nodes and ~~edges~~ do not change, but the supply ~~and demander~~ does change, so the scaling max flow algorithm will take O(log C) time where C = total supply of blood.

b) ~~The two~~ An allocation~~s~~ that would satisfy maximum possible number of patients is if we give ~~8~~ 45 of the O supply to fulfill all demand for O, 5 of the remaining O supply and 36 of the A supply to fulfill 41 units of A's demand, 8 of B's supply to fulfill 8 units of B's demands, and 3 units of AB to fulfill AB's supply. ~~This~~ Only one unit of demand of A remains; ~~which cannot be fulfi~~ There will always be 1

patient left without blood, because the max flow is value is 99 ~~We can verify~~ if we draw a graph as described in part a. We can verify this by the fact that the cut $A = \{ s, \emptyset x_{AB}, x_B, y_{AB} \}$ and $B = V - A$, has capacity 99, while total patients are 100. An explanation to clinic admins would be that that the demand for A and B is ~~9~~ 42 + 45 = 87, and they can only get their supply from O or A, ~~(for A) or only O (for O)~~ but the total supply of O and A is 50 + 36 = 86, so at least one patient of type O or A must not receive blood.

10) ~~We~~ Let $e^* = (v, u)$. First we check if ~~e~~ $f(e^*) < c_{e^*}$. If it is then reducing c_{e^*} by 1 ~~&~~ still keeps the given max flow as valid as capacity conditions are still met, and so our max flow has not changed. Else, if $f(e^*) = c_{e^*}$ then with the new capacity c'_{e^*}, $f(e^*) = c'_{e^*} + 1$, so the capacity and conservation conditions is ~~are~~ no longer met and the previous max flow is invalid. Since $f(e^*) > 0$, $f^{in}_v > 1$ ~~and~~ and $f^{out}_u > 1$. Also, positive flows are acyclic.

traverse

Thus, we go ~~backwor travel~~ throug backward from v, ~~on some~~ (i.e. ~~if~~ for some edge (w,v) ~~in~~ with positive flow, we go from v to w) to eventually get to s_ϕ. Similarly due to acyclicity of positive flows, we can traverse through ~~some~~ positive flow ed forwarde edges from u to t. ~~Appending these~~ eventually get to t. Appending these ~~e~~two sets of edges, which give an s-~~e~~v and u-t path, with e^* in the middle; gives us an s to t path including e^*. If we edit the ~~f~~original flows for these edges to ~~eg~~ be 1 less than ~~b~~before ~~ϕ~~ to get a new flow f', f' now satisfies the capacity conditions ~~and~~ ($f'(e^*) = c'e^*\phi$) and the conservation conditions ($fin_v = fout_v$ $\forall$ v on the edited path). We had to edit an entire path because simply reducing $f(e^*)$ alone would have fixed the capacity condition that had arosen, but invalidated the conservation conditions so the flow would still have been invalid.

~~Sine~~ We know f' is a valid flow, but we do not know if it is a max flow. If we check the residual graph $G_{f'}$, if there are no s-t augmenting paths,

then f' is a max flow and we are done. Else, we augment an s-t augmenting path once to get f'', which is a max flow because $v(f'') = v(f') + 1 = v(f)$, and changing the capacity of a single edge by 1 could have ~~change~~ decreased the max flow value by at most 1.

The running time is $O(m+n)$ because ~~we~~ creating f' takes time $O(m)$, then creating the residual graph $G_{f'}$ takes $O(m+n)$ time, ~~and a single augmentation takes $O(m+n)$~~ finding an augmenting path takes $O(m+n)$ time and then augmenting once takes $O(n)$ time.

11) The statement is false i.e. the negation is true. Let b be any number greater than 1. If b is not an integer we simply round it up to the nearest integer. Now create a graph as follows: We create $b+1$ nodes $u_1, u_2 \cdots u_{b+1}$ and $b+1$ nodes $v_1, v_2 \cdots v_{b+1}$ such that there is an edge ~~from~~ to the sink ~~s to t from~~ source ~~each~~ v_i and an edge ~~to the sink t from~~ each u_i from the source ~~s~~. Suppose

11) The statement is false i.e. the negation is true. Suppose b is any number such that $b > 1$. Now if b is a real number, we round it up to the nearest integer. Then we create a graph as follows: Create b+1 nodes $u_1 \cdots u_{b+1}$. Create b+1 nodes $v_1 \cdots v_{b+1}$. Create an edge from a source s to each u_i and an edge from each v_i to the sink t. Also create an edge from each u_i to v_i and an edge from each v_i to u_{i+1}. All edges must have capacity 1. Suppose the first s-t path considered for augmentation is $s \to u_1 \to v_1 \to u_2 \to v_2 \to \cdots \to u_b \to v_b \to u_{b+1} \to v_{b+1} \to t$. Augmenting this path will increase the flow by 1, and there will be no s-t paths left in the residual graph because all $u \to v$ edges have been saturated. But the maximum flow is b+1 by pushing flow of 1 through every path of the form $s \to u_i \to v_i \to t$. Clearly $1 \leq b+1/b = 1 + 1/b$. Thus we have proved the negation i.e. $\forall\, b > 1$, $\exists$ an instance of the max flow problem such that the forward edge only algorithm is not guaranteed to find a flow of value at least $1/b \times$ max flow value.

13) We construct G' ~~as follows.~~ by replacing each $v \in V$ with v_{in} and v_{out}, joined by an ~~ext~~edge (v_{in}, v_{out}) of capacity c_v. We replace every edge (u,v) by (u_{out}, v_{in}) of capacity $(\sum_{x \in V} c_x) + 1$. Then calculate a max flow in G'.

~~G~~Let f be any flow in G. Let f' be f except since it is in G', let $f(u,v) = f'(u_{out}, v_{in})$. ~~$= f'(v_{in}, v_{out})$~~ Flow conservation holds, ~~because $f(u,v)$~~ and the latter edge respects its capacity because $f^{in}_v \leq c_v$. Conversely any flow in G' gives a flow in G by copying the

Let f be a flow in G. Put $f(u,v)$ on (u_{out}, v_{in}) and put f^{in}_v on (v_{in}, v_{out}). Flow conservation holds, and the latter edge respects its capacity because $f^{in}_v \leq c_v$. Conversely any flow in G' gives a feasible flow in G by copying the flow on the edges (u_{out}, v_{in}) onto (u,v). Conservation at v_{in} shows that the flow through v = the flow on $(v_{in}, v_{out}) \leq c_v$. The two flows have the same

Value, hence a max flow in G' gives a max flow in G.

Let an s-t cut in G be a set $B \subseteq V - \{s\}$ and $A = V - B$ such that every s-t path in G has at least one node in B. Its capacity $c(A,B)$ is $\sum_{x \in B} c_x$. Removing edges of the form (v_{in}, v_{out}) from G' for every $v \in B$ seperates the source and the sink. Conversely, a minimum cut in G' does not have any edges crossing the cut that are not of the form (v_{in}, v_{out}), because those edges have capacity $\sum_{v \in V} c_v$, and so it coresponds to an s-t cut of the same capacity in G. The regular Max-Flow Min-Cut theorem gives...

$$\text{max flow value} = \min_X \sum_{v \in X} c_v .$$

Constructing G' takes $O(|V| + |E|)$ time, and using the sceling max flow algorithm on it takes $O(|E|^2 \lg C)$ time where C is the sum of the capacities of edges leaving S_{out}.

a)

15) Create a set of nodes $X = \{x_1 \dots x_n\}$ where x_i represents p_i, and a set of nodes $Y = \{y_1 \dots y_n\}$ where y_i represents d_i. Also add an edge (x_i, y_j) iff d_j & S_i.

If G_1 has a perfect matching M, then for every node v in G_1 Suppose G_1 has a perfect matching M. Then every by definition of perfect matching, every node appears in exactly one edge of M, and our graph's structure ensures that one endpoint is in X and the other in Y. Since edges that could connect a person to a night they cannot cook on are not included, in G_1, we consequently have a feasible dinner structure. Conversely, suppose we have a feasible dinner structure. Then every p_i is assigned to a night d_j such that no other person is assigned to d_j, and since d_j & S_i, we have edge (x_i, y_j) in G_1 and these edges form a perfect matching.

b) We transform the graph G_1 from part a into G' as follows: We add a source s and a sink t, such that there is an edge (s, x) $\forall x \in X$ and an edge (y, t)

$\forall t \in \forall y \in Y$. These edges are directed, and we also make the edges b/w X and Y directed (from the node in X to the node in Y). Additionally every edge has capacity 1. The only exception is edges that th We For each p_i matched to d_b in the given sch schedule. We create a flow f in G' as follows: for each p_a matched to d_b in the schedule, except for p_i, and p_j, ~~d_k and~~ ~~f f(x a,~~ $f(x_a, y_b) = f(s, x_a) = f(y_b, t) = 1$. We now have a flow of value $n-2$; and since the max possible flow is n, we need at most two augmentations to get a max flow. If the max flow has value n, then we have a perfect matching ~~of~~ ~~S~~ and so by part a, we have a feasible schedule. Else there is no feasible schedule.

~~Since~~ Constructing the graph takes $O(m+n)$ time, and there are at most 2 augmenting, and searching for each takes $O(m)$ time and since m is at most $2n+n^2$, each search takes $O(n^2)$ time and so the overall running time is $O(n^2)$.

16) We create a graph G as follows: We create a source s, a sink t, and we create m nodes $A_1 \dots A_m$ to represent each advertisor. We also create n nodes $B_1 \dots B_n$ to represent each user of the site. We add an edge (s, A_i) for every A_i, with lower bound r_i and upper bound n. We also add an edge (A_i, B_j) iff $X_i \cap Y_j \neq \emptyset$, with capacity 1. We also add an edge (B_i, t) with lower bound = capacity = 1 for all B_i. Finally we add an edge from t to s with capacity ∞ n. Then we simply check if a feasible integer flow exists in this graph (which must exists if a feasible flow exists because the edge capacities are integers). If it does, then a good advertising policy as defined exists, and an ad by advertisor i is shown to user j iff edge (A_i, B_j) exist with a flow of value 1. Else a good advertising policy is not possible.

To show correctness, Let f be an integer valued circulation in G. Then each user gets 1 add because each B_i has one outgoing edge with lower bound = capacity = 1, and every advertisor i shows at least r_i ads to its chosen demographics because there is only one incoming edge into A_i with lowerbound r_i, and an edge from A_i

to B_j only exists if ∅ user j is part of advertisor A_i's target demographics. Conversely, if a good ad policy exists, then ~~we c if~~ we put a flow of 1 on (A_i, B_j) where ~~user j~~ advertisor i's ad is shown to user j, a flow ~~of~~ of 1 on all outgoing edge of all B_i (because each user gets 1 ad), and hence a flow of n on (t, s), and for each (s, A_i), we put $f^{out} A_i$, which will be at least r_i, to get a circulation on G_r that respects all capacities, lower bounds and conservation conditions.

Creating G_r takes ~~$\Theta(m+n)$ time~~ $O(|E| + |V|)$ time where $|V| = m+n$ and $|E| = O(mn)$ so overall $O(mn)$ time, and then finding a circulation takes ~~$O(m+n)$ time to create~~ G_r (by adding a super so $O(m+n)$ time to turn G_r into the reduced graph G' and $O(|E|^2 \lg(nm))$ time to ~~calculate~~ = $O(m^2n^2 \lg(mn))$ time to calculate max flow in G' using the parameterized fold fulkerson algorithm. (which serves as a check for whether or not a circulation exists in G_r

Hence overall runtime is $O(m^2n^2 \lg(mn))$

18

18|a| We create G' by editing G as follows:- for each undirected edge in E, make it directed from the node in X to the node in Y, with capacity 1. Add a ~~sink~~source s, with an edge to each node in X and capacity 1, and a sink t with an edge from each node in Y, again with capacity 1. Also, for every $y \in Y$ such that there exists $(x, y) \in M$ for some $x \in X$, set a lower bound of 1 on the edge (y, t). Also create an edge (t, s) with lower bound = capacity = $|M| + k$. Now ~~if~~ check if a feasible integer ~~flow~~ integer circulation exists in G', if it does, then every edge ~~of~~ from a node in X to a node in Y ~~that is saturated~~ with ~~with~~ a flow of 1 is in M'. Else no ~~mated~~ solution exists.

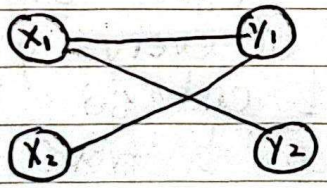
Suppose a feasible integer ~~flow~~ f integer circulation f exists in G'. Then the flow out of every $y \in Y$ that is matched in M is 1 and so there is an incoming flow of 1 from a node x in X to y, and since there is a flow of $|M| + k$ from t to s, there is a flow of 1 into t from $|M| + k$ nodes in Y each with a node x in X they are matched with. Conversely, if we have a solution M', then we can set a flow of 1

$\in Y$

on each (y,t) where y is matched in M', a flow of ~~$|M|+k$~~ $|M'|=|M|+k$ on (t,s), a flow of 1 on every (s,x) where $x \in X$ is matched in X, and a flow of 1 on every edge $e'=(x,y)$ $\in M'$ where $x \in X$ and $y \in Y$. This gives a feasible integer circulation in G'.

Creating G' from G takes $O(m+n)$ time, and then reducing G' to G'' we can use for finding a feasible circulation will take $O(|M|)$ time (we add an edge from an edge from a super source or sink to nodes with lower bounds on an adjacent edge, and there are $O(|M|)$ such nodes). Calculating a max flow in G'' using Ford Fulkerson then takes ~~$O(mk)$ time~~ $O(k(m+n))$

b) Let $k=1$, $M=\{(x_1,y_1)\}$ and $G=$



The only solution is $M'=\{(x_1,y_2),(x_2,y_1)\}$ and $M \not\subseteq M'$.

c) Let M'' be some maximum matching in G such that not every $y \in Y$ covered by M is covered by M''. Now we create a new graph $G' = (V, E')$ where $E' = (M \setminus M'') \cup (M'' \setminus M)$.

Then every connected component of G' is an alternating path or cycle. Suppose some $y \in Y$ is covered by M but not M'', then y is the endpoint of an alternating path P, whose first edge is from M. P cannot contain one more edge from M than from M'', because then we could replace the M''-edges of P with its M edges to get a larger matching than M'' i.e. a contradiction.

Thus P contains equal amounts of M and M'' edges. Because P starts at a vertex $y \in Y$ covered by M edge, it ends with a vertex $y' \in Y$ with an M'' edge (y' is not covered by M because then P would not end at y').

Now let $M''' = (M'' \setminus E(P)) \cup (M \cap E(P))$

Then M''' is a matching such

that $|M'''| = |M''|$ but M''' covers y that M'' did not and uncovers y'' that M did not cover anyway.

Repeating this process for every $y \in Y$ that is covered by M but not M'' will thus give a matching with the same size as M'' (i.e. K_2) but covers every $y \in Y$ covered by M_1 and so $K_1 = K_2$

19) a) Create a graph G as follows:- We create a source s, a sink t, nodes $d_1 \cdots d_n$ for ~~every~~ each day, and nodes $m_1 \cdots m_k$ for each doctor. Then we add an edge (s, d_i) for every d_i and with capacity ~~P_i~~ 1, an edge (d_i, m_j) with capacity 1 iff $d_i \in L_j$ and an edge (m_i, t) for every m_i with capacity $|L_i|$.

Now we find a max flow f in G; ~~and if the flow has value~~

Suppose

Suppose f ~~is~~ has value $\sum_{i=1}^{n} P_i$. ~~ie~~ Then every ~~edge~~ d_i has ~~a~~ a flow of 1 out on P_i out edges, and since an out edge only exist to m_j iff $d_i \in L_j$, that means ~~d_i~~ is all d_i that have an edge with flow 1 to m_j form L'_j. ~~&~~ with condition A ~~and~~ B fulfilled. ~~Similarly since~~

Conversely, if

We have lists $L'_1 \dots L'_k$ satisfying properties A and B, we can put a flow of 1 on (d_i, m_j) iff $d_i \in L'_j$, ~~a flow of~~ and ~~by proper~~ ~~since~~ d_i by property B, there are p_i such ~~out~~ edges and so $f^{out} d_i = p_i$ and so to balance it we put flow p_i on (s, d_i). ~~Since for each~~ Now we ~~balance~~ fulfill the conservation conditions at each m_i by setting flow on (m_i, t) to $f^{in} m_i$, and we have a valid flow of value $\sum_{i=1}^{n} p_i$ in G.

Thus if $v(f) = \sum_{i=0}^{n} p_i$, then L'_i is all the days j such that $f(d_j, m_i) = 1$, ø else such a set of lists do not exist.

~~Creating G takes $O(\sum p_i)$ $O(n+k)$ $O(K)$ time where $K = \sum_{i=0}^{k} |L_i|$~~

$d > k$

Creating G takes $O(n+k+K)$ time, where $K = \sum_{i=1}^{k} |L_i|$. Using the Ford Fulkerson algorithm to then find a max flow takes $O(KP)$ time where $P = \sum_{i=1}^{n} p_i$.

b) Create G as follows: create a source s, a sink t, for each doctor i, a node m_i and m_i', and for each day i a node

d_i. For each m_i, create edge (s, m_i) with capacity $|L_i'| + c$ and edge (m_i, m_i') with capacity c. Also, for each m_i and d_j, add (m_i, d_j) iff day $j \in L_i$, with capacity 1, and ~~add~~ ~~(m_i', d_j)~~ for each m_i' and d_j, add (m_i', d_j) iff day $j \notin L_i$, with capacity 1. Also add (d_i, t) for each d_i with capacity p_i. Now find a max flow f in G.

If ~~f~~ Let $P = \sum_{i=1}^{n} p_i$. If f has value P then every day i has an edge coming into it from p_i different doctors (m_i) and each doctor i is assigned to at most c days not in L_i because $f^{out}(m_i') \leq c$, and so L_i' is the set of every day j such that an edge (m_i, d_j) or (m_i', d_j) exists with flow of 1. Conversely if lists $L_1' \ldots L_n'$ exist, we can create a flow of value P by putting a flow of 1 on (m_i', d_j) or (m_i, d_j) for every day j in L_i, and ~~so since $|L_i'|$ so we can fulfill conserv~~ so by property B we can fulfill Conservation condition at d_j by putting flow p_i on each (d_j, t). ~~Similarly~~ Finally we can put flow $f^{out} m_i$ on (s, m_i) for each m_i.

There are ~~O(kn)~~ ~~kn edges~~ kn + 2n + k edges i.e. O(kn) edges and 2n+k+2 nodes i.e O(n+k) nodes so creating G takes O(kn) time, and then running Ford Fulherson algorithm to find a max flow takes O(knP) time hence overall running time is O(knP).

23) Compute a max flow f in G. In the residual graph Gf, do a BFS rooted at s, all nodes ~~re~~ that can be reached are upstream. Then do a BFS rooted at t, but traverse edges in their opposite direction. All reachable nodes are downstream. The remaining nodes are central.

~~This The~~ Let In every minimum s-t cut, (A,B), there should be no edges in the residual graph Gf that crosses from A to B, because if there was then c(A,B) > v(f) which contradicts the fact that (A,B) is a minimum cut. Hence the nodes reachable from s in Gf are all ~~and in~~ every A, ~~upstream nodes, because the re~~ hence they represent all upstream nodes.

Similarly, every node that can reach t is in every $B_{\phi,}$ and hence represent all downstream nodes. It follows that the remaining nodes are central.

Calculating f takes time $O(mC)$ and computing the two BFS / DFS take $O(m+n)$ time, hence $= O(m)$ because G is connected hence $m = O(n)$, and so the overall runtime is $O(mC)$

29) Create a graph $G = (V, E)$ as follows:- one source s, one sink t, and a node app_i for each software app i. Create an edge $(s, app1)$ with capacity $\sum_{i=1}^{n} b_i + 1$, an edge (app_i, t) with capacity b_i, for each software $i \neq 1$ and an edge (app_i, app_j) with capacity π_{ij} for each pair of apps i and j. Now compute a max flow f. The set of vertices reachable from s in the residual graph G_f represent apps that stay on the old system, and the rest move to the new system. (i.e S consists of all the apps represented by nodes not reachable

where A is source side (A, B)

from s). Our algorithm works because this is a minimum cut, and every edge passing the source side to the sink side is either of form (app_i, app_j) thus representing the penalty of two apps in different systems via its capacity, or of the form (s, app_i) thus representing b_i for an app in the old system. We have thus minimised :-

$$\sum_{i \in A} b_i + \sum_{\substack{(i,j) \in E : \\ \{i,j\} \cap A = 1}} x_{ij}$$

And so we have maximised the quantity:-

$$\sum_{i \in B} b_i - \sum_{\substack{(i,j) \in E : \\ \{i,j\} \cap A = 1}} x_{ij}$$

Moreover, we have set the capacity of (s, 1) sufficiently large to make sure it cannot be saturated with flow thus guaranteeing app 1 stays on the old system.

Creating G takes $O(n^2)$ time and finding a max flow using Ford Fulkerson will take $O(n^2 C)$ time where $C = \sum_{i=1}^{n} b_i + 1$ and so the overall running time is $O(n^2 C)$, or $O(n^4 \lg C)$ if we use the scaling version of Ford Fulkerson (which is true polynomial rather than pseudopolyno

a)

39) Create a graph $G = (V, E)$ as follows: Create a source s, a sink t, a node R_i for each row i, a node C_j for each column j. For each R_i, create edge ~~$f R_i, \in$~~ (s, R_i) with capacity equal to the sum of all entries in that row. For each ~~#~~ pair R_i, C_j, create edge (R_i, C_j) with capacity 1. And for each C_i, create (C_i, t) with capacity equal to the sum of all entries in that row. Compute a max flow, if it has value equal to the sum of every entry in the matrix, the desired rounding is possible. ~~This is because All~~ capacities being integers gurantees an integer flow exists; ~~and by flow conservation we know~~ A valid rounding coresponds exactly to an integer flow saturating all edges out of s, since each (R_i, C_j) carries ~~flow~~ 0 or 1, representing the rounded entry, while flow conservation preserves every row and column. Conversely, any such integral max flow gives a valid rounding

b) Keep G as in part a, but for each ~~$(C_i$~~ (R_i, C_j), set 'a

lower bound of $\lfloor x \rfloor$ and capacity of $\lceil x \rceil$, where x is the entry at (i, j). Also add edge (t, s) with lower bound equal to capacity which is sum of all entries. Now rounding is possible if a feasible ~~flow exists~~ circulation ~~exists~~ (and hence feasible integral circulation) exists. A feasible integral circulation assigns each (R_i, C_j) either $\lfloor x \rfloor$ or $\lceil x \rceil$ where x is the entry (i, j). Flow conservation preserves all row and column sums. Conversely any valid rounding defines such a circulation.

c)